

PORTADA

Análisis y Diseño de Sistemas II

Semana 3 — Modelado Conceptual: Beneficios en el Marco del Desarrollo de Software

Universidad de Antioquia · Ingeniería de Sistemas



ENCUADRE

Arranca la Unidad 2 — Modelo de Análisis

Con el equipo, el alcance y el Sprint 0 ya montados, esta unidad construye el puente entre **"lo que el negocio necesita"** y **"lo que el sistema va a construir"**. Tres semanas, tres artefactos:

Cuándo	Qué se construye
Semana 3	Modelo de dominio (elaboración y refinamiento)
Semana 4	Diagrama de clases UML + Diagramas de Secuencia del Sistema (DSS)

Hoy arrancamos por la pregunta previa: **¿por qué invertir tiempo en modelar antes de programar?**



APERTURA

5 años

170 millones de dólares
Cero líneas en producción



CASO · FICHA TÉCNICA

FBI Virtual Case File (VCF)

Organización	Federal Bureau of Investigation (FBI), Estados Unidos
Contratista	Science Applications International Corporation (SAIC)
Objetivo	Reemplazar el sistema de casos en papel/formularios por un sistema digital de gestión de casos e investigaciones
Periodo	Programa Trilogy, iniciado en 2000-2001; VCF cancelado en 2005
Inversión reportada	Cerca de 170 millones de dólares
Desenlace	Proyecto cancelado sin entrar en producción; reemplazado años después por el programa Sentinel

Caso ampliamente documentado en informes de la Oficina del Inspector General del Departamento de Justicia de EE. UU. y citado en textos clásicos de Ingeniería de Software.

CASO · CONTEXTO

Un problema real, urgente — y mal entendido

Tras los atentados del 11 de septiembre de 2001, el FBI necesitaba con urgencia dejar de depender de archivos en papel para correlacionar información entre investigaciones. La presión por entregar rápido era enorme.

El equipo de desarrollo empezó a **construir pantallas y código** casi en paralelo con la definición de qué era exactamente un "caso", un "expediente" o una "pista" para los agentes del FBI — sin un modelo conceptual estable que fijara ese vocabulario antes de programar.



CASO · LO QUE FALLÓ

Sin modelo compartido, todo se vuelve movable

- Agentes, analistas y desarrolladores usaban las mismas palabras ("caso", "pista", "expediente") con significados distintos entre sí.
- Cientos de solicitudes de cambio llegaron durante el desarrollo, muchas de ellas producto de ambigüedades que un modelo conceptual habría expuesto desde el principio.
- El código se escribió antes de que el equipo tuviera acuerdo sobre las entidades centrales del dominio y cómo se relacionaban entre sí.



TRANSICIÓN

¿Qué habría evitado esto?

No un mejor lenguaje de programación, ni más horas de código: un **modelo conceptual del dominio**, acordado antes de construir, que respondiera con precisión preguntas como "¿qué es un caso?", "¿un caso puede tener varias pistas?", "¿quién puede cerrar un caso?".

Eso es exactamente lo que esta unidad enseña a construir.



CONCEPTO

Qué es un modelo conceptual

Un **modelo conceptual** es una representación de los conceptos, términos y relaciones más importantes de un dominio del mundo real — construida para lograr que todos los interesados (negocio, analistas, desarrolladores) compartan el mismo entendimiento antes de diseñar o programar.

- No describe software: describe el **dominio del problema**.
- No tiene botones, pantallas ni bases de datos.
- Se expresa típicamente como un **modelo de dominio** (el tema del resto de esta semana).

CONCEPTO

Los beneficios, uno por uno

Beneficio	Qué previene
Vocabulario compartido	Que "caso" signifique una cosa para el negocio y otra para el desarrollador
Detección temprana de errores	Diseñar y programar sobre una idea del dominio que era incorrecta desde el inicio
Base para el diseño	Empezar a codificar clases y tablas sin saber qué entidades existen ni cómo se relacionan
Trazabilidad	Perder el hilo entre un requisito del negocio y la parte del sistema que lo implementa

CONCEPTO

El costo de corregir crece con el tiempo

La intuición detrás de modelar temprano: un malentendido sobre el dominio detectado en una conversación de análisis cuesta una corrección de texto. El mismo malentendido detectado después de programar cuesta reescribir código, pruebas y, a veces, datos ya en producción.

Requisitos/Análisis → Diseño → Código → Pruebas → Producción

(el costo de corregir crece en cada etapa)



CONCEPTO EN DISPUTA

Honestidad sobre esa curva

Afirmación clásica: corregir un defecto de análisis cuesta muchísimo más en producción que en la etapa de requisitos.

Advertencia: los multiplicadores concretos que suelen citarse (ej. 1 : 10 : 100) **no tienen respaldo empírico sólido y uniforme** — varían mucho según el proyecto, el dominio y la tecnología.

La idea general —modelar antes reduce sorpresas costosas después— sí se sostiene; los números exactos de la curva, no hay que citarlos como ley universal.

CONTRAEJEMPLO

Lo que el modelado conceptual NO es

- **No es** un mockup o boceto de pantallas — eso es diseño de interfaz.
- **No es** un esquema de base de datos — eso viene después, en diseño/persistencia.
- **No es** un diagrama de clases de software con métodos y visibilidad — eso es diseño orientado a objetos.

El modelo conceptual vive antes que todo eso: describe el dominio, no la solución de software.



TENSIÓN CONCEPTUAL

¿Modelar antes choca con ser ágil?

Postura 1 — "Just enough analysis": Scrum y el desarrollo ágil desconfían del "Big Design Up Front"; prefieren descubrir el dominio incrementalmente, historia a historia.

Postura 2 — "Sin modelo, no hay con qué conversar": incluso un Sprint corto necesita que el equipo se ponga de acuerdo en qué significan sus propias historias de usuario.

La resolución que sigue este curso: un modelo de dominio ligero, que se refina en cada Sprint — no un documento congelado de 100 páginas antes de escribir la primera línea de código.



CONCEPTO

Dónde encaja en lo que viene

El modelo conceptual de esta semana no es un ejercicio aislado: es el insumo directo de todo lo que sigue en la Unidad 2 y en la Unidad 3 (arquitectura):

1. **Modelo de dominio** (miércoles y viernes) — las clases conceptuales y sus relaciones.
2. **Diagrama de clases UML** (semana 4) — la notación formal del mismo modelo.
3. **Diagramas de Secuencia del Sistema** (semana 4) — cómo el sistema, ya entendido como caja negra, reacciona a eventos externos.

INTERACCIÓN

Encuentren su propia ambigüedad

En equipos, sobre el enunciado del problema de su proyecto de Fábrica Escuela:

1. Identifiquen un término que usen todo el tiempo (ej. "pedido", "usuario", "solicitud", "reserva").
2. Pregúntense: ¿todos en el equipo lo definirían exactamente igual, con los mismos atributos y límites?
3. Anoten un término donde encontraron desacuerdo o ambigüedad.

5 minutos · compartan un ejemplo en el chat

CONCEPTO · CIERRE DE LA APERTURA

Volviendo al caso del FBI

El VCF no falló por falta de talento técnico ni de presupuesto: falló porque el equipo empezó a construir sin haber fijado, con precisión, los conceptos del dominio que el sistema debía representar.

Eso es exactamente lo que el modelo de dominio de esta semana busca evitar en su propio proyecto — antes de llegar a un punto donde corregirlo sea caro.

SÍNTESIS

Lo que hay que llevarse de hoy

1. Un **modelo conceptual** describe el dominio del problema, no el software.
2. Sus beneficios: vocabulario compartido, detección temprana de errores, base para el diseño y trazabilidad.
3. El caso del FBI VCF (~170 millones de dólares, cancelado sin producción) es un ejemplo real de lo que cuesta construir sin ese acuerdo previo.
4. Modelar no es incompatible con ser ágil: el modelo se mantiene ligero y se refina Sprint a Sprint.
5. El modelo de dominio (miércoles/viernes) es la forma concreta que toma el modelado conceptual en este curso.



CIERRE

Para el miércoles

Traigan a la sesión del miércoles el **enunciado del problema y las historias de usuario** ya definidas de su proyecto — sobre eso construimos, en vivo, el primer modelo de dominio de cada equipo.

